

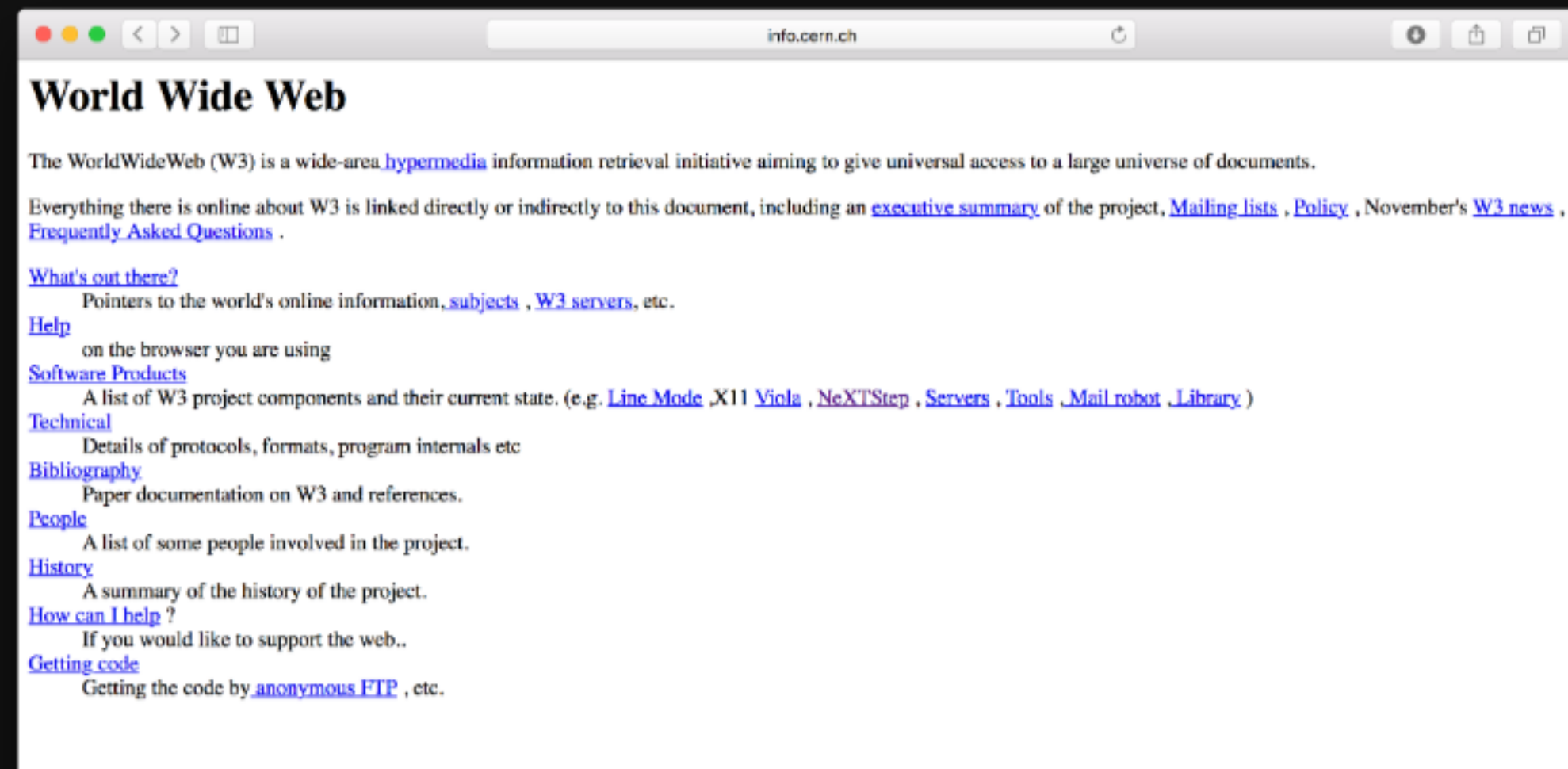
JavaScript Frontend Web Development



The Web Platform

World Wide Web

*A web of hypertext documents to be viewed by
browsers using a client-server architecture* Tim Berners Lee

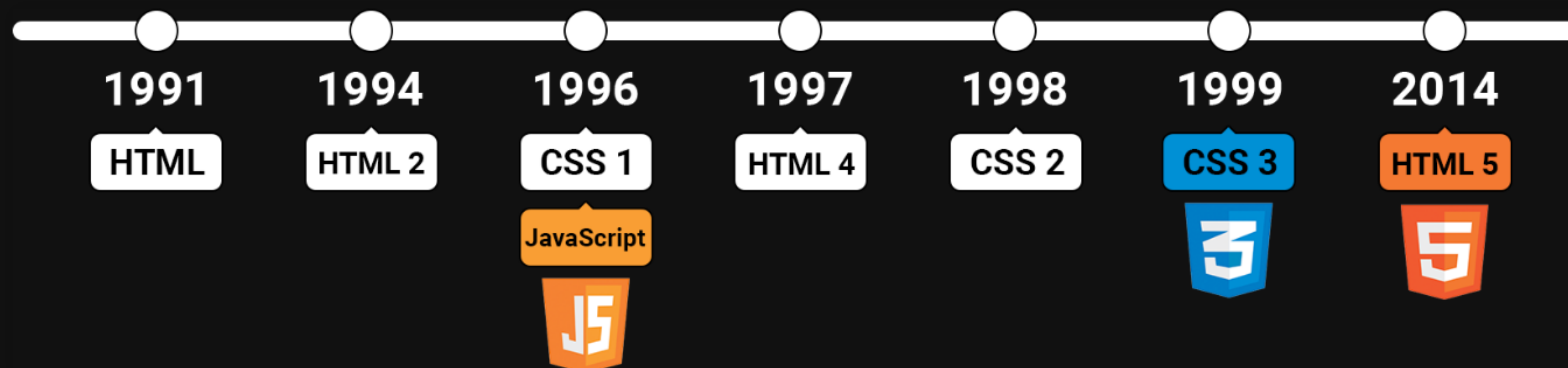


Modern Web



The modern Web brought us a plethora of new technologies, such as:

- **CSS** to describe the appearance or presentation of content on a webpage
- **JavaScript**, the programming language that runs in the browser and can add interactivity and other dynamic features to web applications
- **Web APIs**, used to perform a variety of tasks, such as manipulating the **DOM**, playing audio or video, or generating 3D graphics
- **AJAX**, used to fetch data from a server without having to do a full page refresh
- **WebRTC**, enabling audio/video streaming and data sharing between browser clients

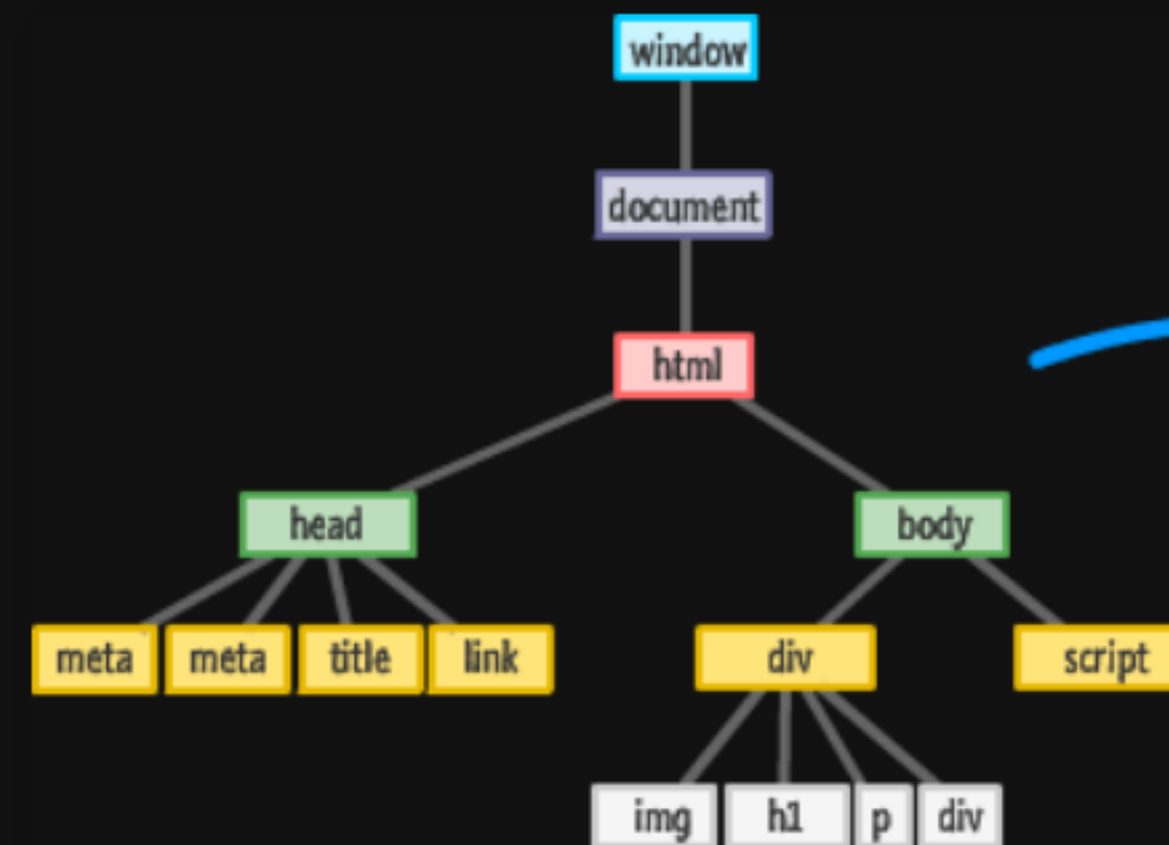


Document Object Model

Document Object Model



Language neutral interface that allows dynamic access and update to the contents, structure and style of web pages



The DOM



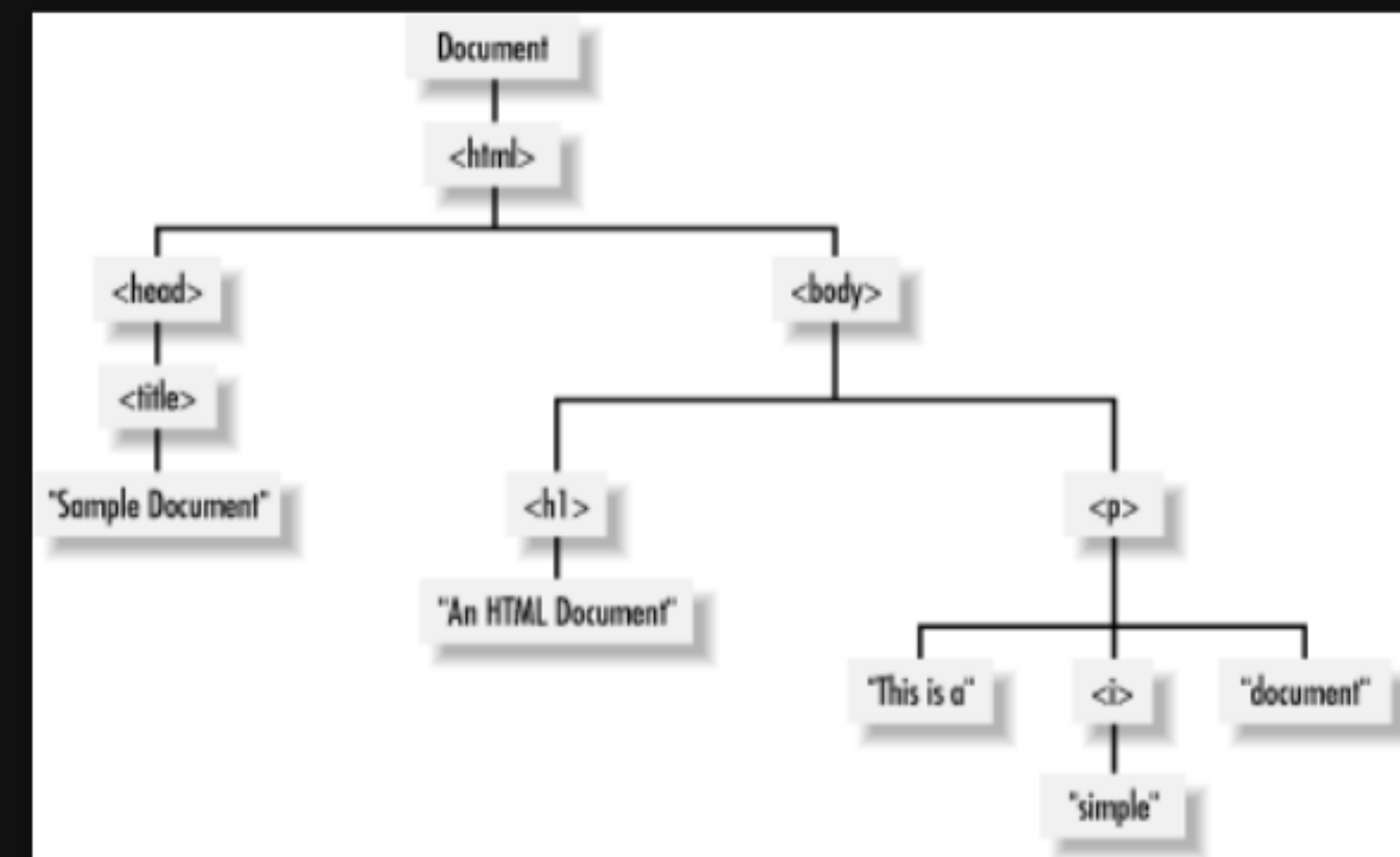
The Browser
(aka what you see)

It is an object oriented representation of the web page and can be modified with JavaScript

DOM Tree

In the DOM, documents assume a tree structure of nodes

Browsers use layout engines such as Webkit (Safari), Blink (Chrome) or Gecko (Firefox) to parse HTML into a DOM and render it on screen



These engines historically implemented the DOM standards to varying degrees of compliance, causing inconsistencies in the way the DOM is accessed in different browsers

How do I access the DOM?

The window object represents a browser window containing a DOM

The document property points to the DOM loaded in that window

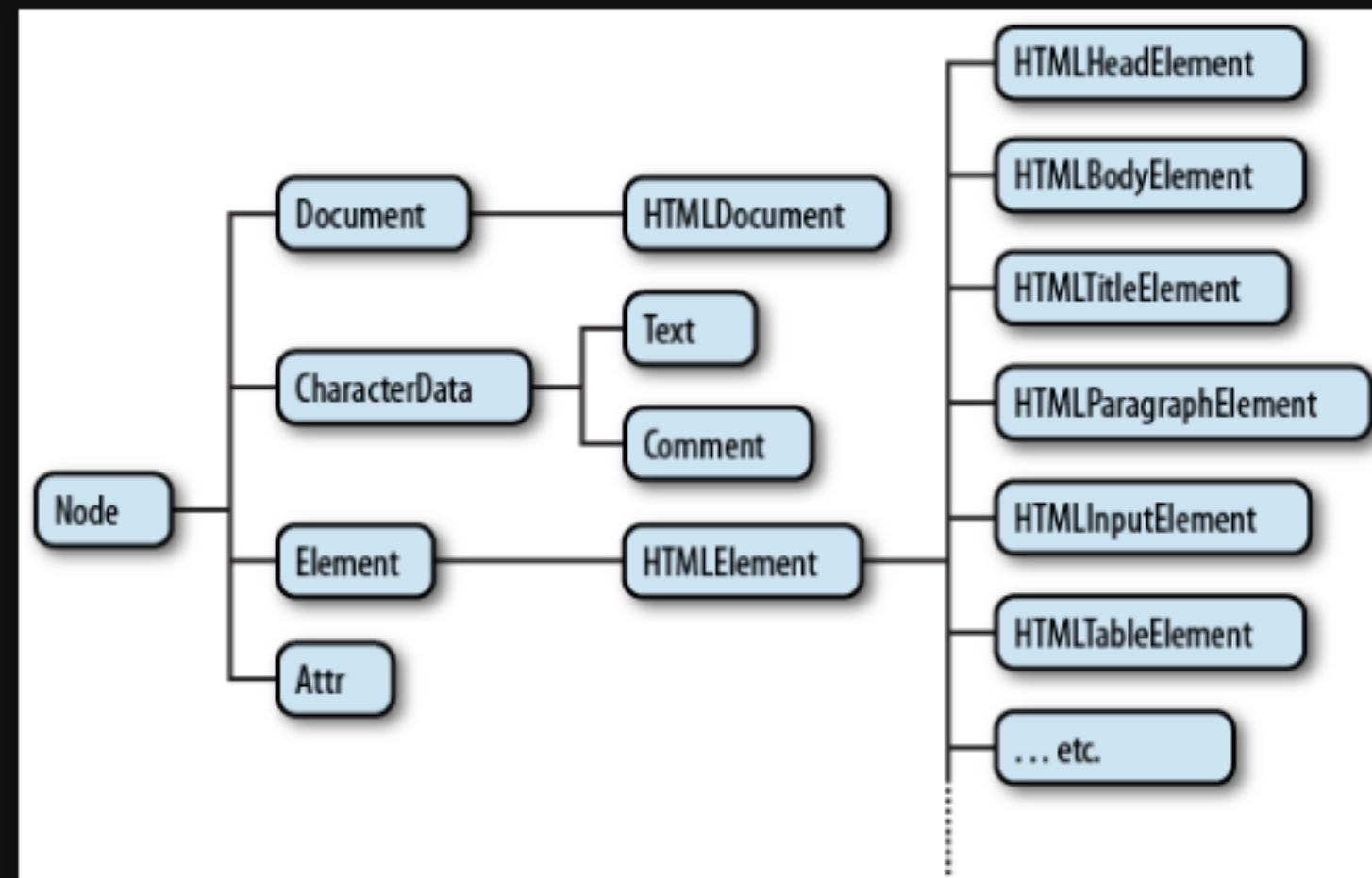
```
<html>
  <head>
    <script>
      // create a couple of elements in an empty HTML page
      window.onload = function() {
        var heading = document.createElement("h1");
        var heading_text = document.createTextNode("Academia de Código");
        heading.appendChild(heading_text);
        document.body.appendChild(heading);
      }
    </script>
  </head>
  <body>
  </body>
</html>
```

The window.onload method receives as argument a function callback to be invoked when the document is loaded into the browser window. The DOM may not fully populated before that!

DOM Objects

In the DOM API, each HTML element is represented by an object

Each object has various properties and methods that correspond to the appearance or behavior of the corresponding HTML element



Each object can have parents, children and siblings

The Document Object

The document node represents the HTML document itself and it is at the root of the DOM tree

It provides methods for creating new nodes in the document as well as searching for existing ones

- `document.createElement(name)` - create an element of the given tag name
- `document.getElementById(id)` - get the element with the given id
- `document.getElementsByClassName(class)` - get a list of elements for the given class name
- `document.getElementsByTagName(name)` - get a list of elements for the given tag name

```
var elem = document.getElementById("hello");  
<p id="hello">Hello world!</p>
```

Create content on the DOM

Starting with a basic HTML table:

```
<table id="users-table">
  <tr>
    <th>Name</th>
    <th>Email</th>
    <th>Edit</th>
    <th>Delete</th>
  </tr>
</table>
```

This is what is required to populate it with users data:

```
// get the table's DOM object
var usersTable = document.getElementById('users-table');

// create a new table row, added to the end of the table
var row = usersTable.insertRow(-1);

// modify the table row HTML
row.innerHTML = htmlStr;
```

Traversing the DOM

After retrieving a DOM element, there are a number of properties that can be used to get elements from elsewhere in the tree

- `element.parentNode` - Returns the parent node of the given element
- `element.childNodes` - Returns a collection of child nodes of the given element
- `element.firstChild` - Returns the node's first child in the tree, or null if the node is childless
- `element.lastChild` - Returns the node's last child in the tree, or null if the node is childless
- `element.nextSibling` - Returns the node immediately following the specified one in its parent's `childNodes` list, or null if the specified node is the last node in that list
- `element.previousSibling` - Returns the node immediately preceding the specified one in its parent's `childNodes` list, null if the specified node is the first in that list

Element Attributes

Most HTML attributes of an element can be accessed through the object property of the same name

```
var elem = document.getElementById("logo_image");
console.log("The URL of this image is: " + elem.src);

// replace the old logo on the page with a new one
elem.src = "../img/new_logo.jpg";

// apply some CSS to the new logo
elem.style = "max-height: 50px; border: 1px solid"
```

Element Contents

Element contents can be retrieved or changed

- `element.innerHTML` - This property represents the HTML within the element
- `element.textContent` - This property represents the plain text within the element. This is equivalent to the inner HTML, but with all the tags stripped out

```
var button = document.getElementById('submit');  
button.textContent = 'DO NOT PRESS!!!';
```

{ Exercise }

Head to www.google.com and apply some creative CSS and content to the logo and sign-in button

Events

The DOM provides methods for registering and unregistering event listeners on elements

EventTarget is an interface implemented by DOM nodes that can receive events and may have listeners for them

- `element.addEventListener(type, callback, options)` - Register an event handler of a specific event type on the EventTarget
- `element.removeEventListener(type, listener, options)` - Removes an event listener from the EventTarget
- `element.dispatchEvent(event)` - Dispatch an event to this EventTarget

```
var helloWorldFn = function() {  
    console.log("hello world!");  
};  
  
var btn = document.getElementById("hello_world_btn");  
btn.addEventListener("click", helloWorldFn, false)
```

Event Default Action

Many events have a default action

Clicking on a hyperlink will make the browser navigate to the target page by default

Clicking on a form submission button will cause the browser to submit the form to the server by default

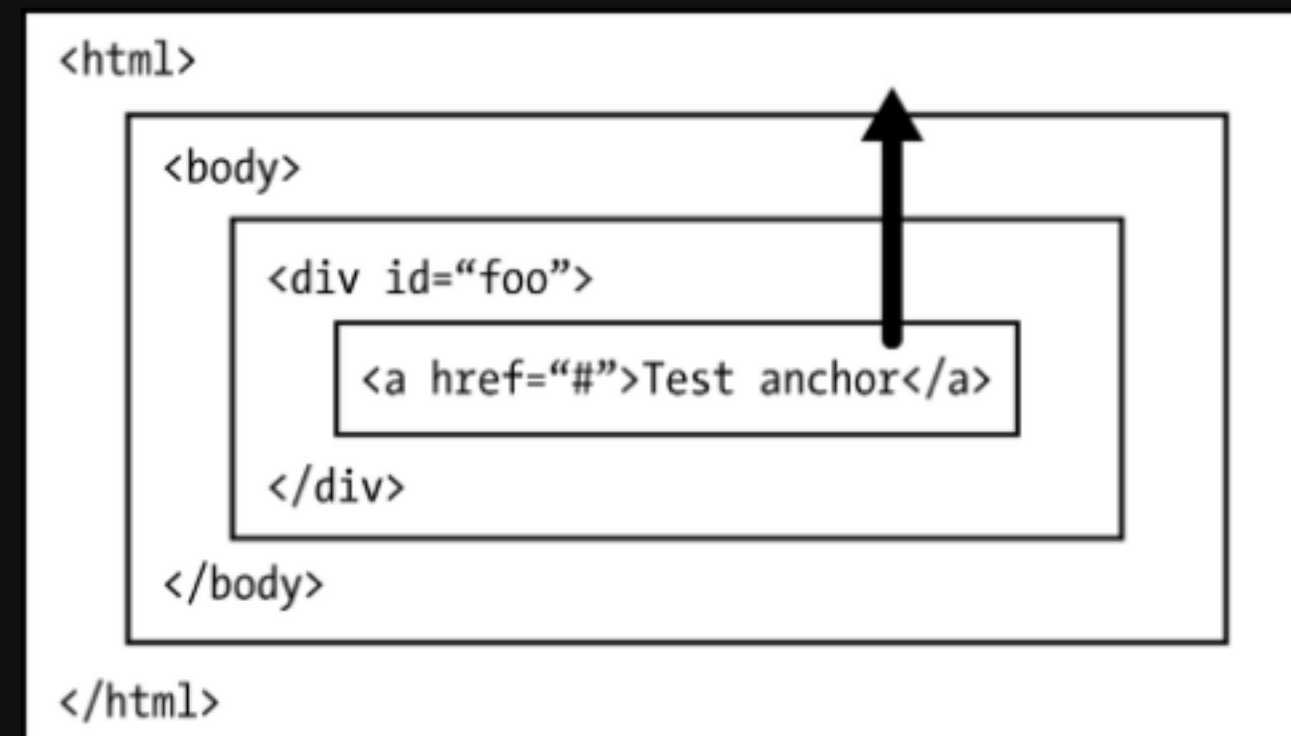
If there is an event listener attached to an element, the event listener will fire first before the default browser action is run.

```
var preventDefault = function(event) {  
    event.preventDefault()  
};  
  
var link = document.getElementById("disabled_link")  
link.addEventListener("click", preventDefaultFn, false);
```

The event listener can choose to prevent the action from being run

Event Propagation

When an event is fired, the element's event handlers are first fired.
The element's parent's event handlers are fired next.
That element's parent's event handlers are also fired, and so on, until the dom root element (**document**) is reached

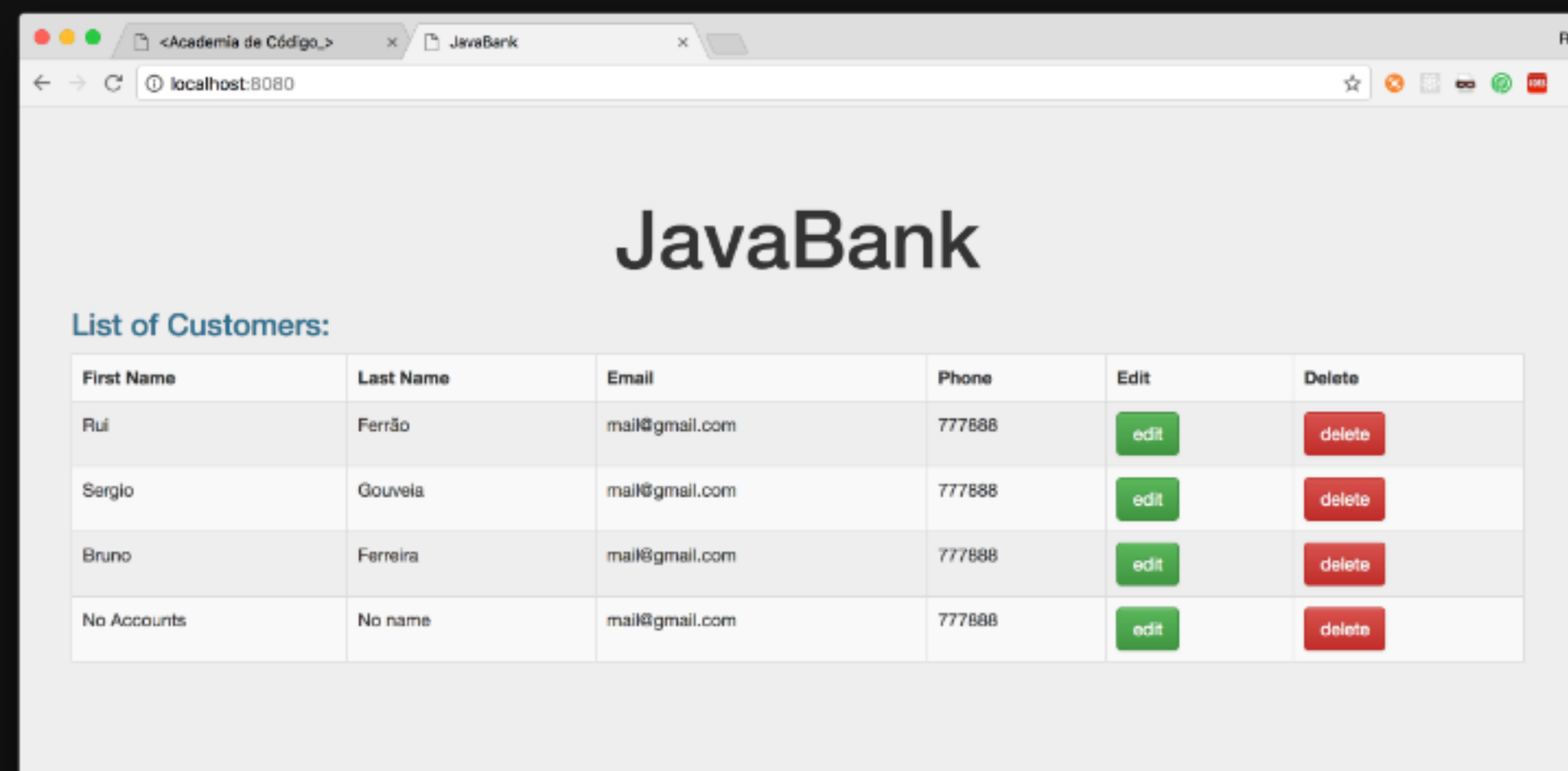


This behaviour is called Event Bubbling

When an event handler is fired, it has an opportunity to stop the event from propagating to its parent. To do so, the handler can call event.stopPropagation() method on the event object

{ Exercise }

List Customer data using the DOM API



First Name	Last Name	Email	Phone	Edit	Delete
Rui	Ferrão	mail@gmail.com	777888	edit	delete
Sergio	Gouveia	mail@gmail.com	777888	edit	delete
Bruno	Ferreira	mail@gmail.com	777888	edit	delete
No Accounts	No name	mail@gmail.com	777888	edit	delete

```
var customerData = [  
  {"id":1,"firstName":"Rui","lastName":"Ferrão","email":"rui@gmail.com","phone":"777888"},  
  {"id":2,"firstName":"Sergio","lastName":"Gouveia","email":"sergio@gmail.com","phone":"777999"},  
  {"id":3,"firstName":"Bruno","lastName":"Ferreira","email":"bruno@gmail.com","phone":"777666"},  
  {"id":4,"firstName":"Rodolfo","lastName":"Matos","email":"rodolfo@gmail.com","phone":"777333"}  
];
```

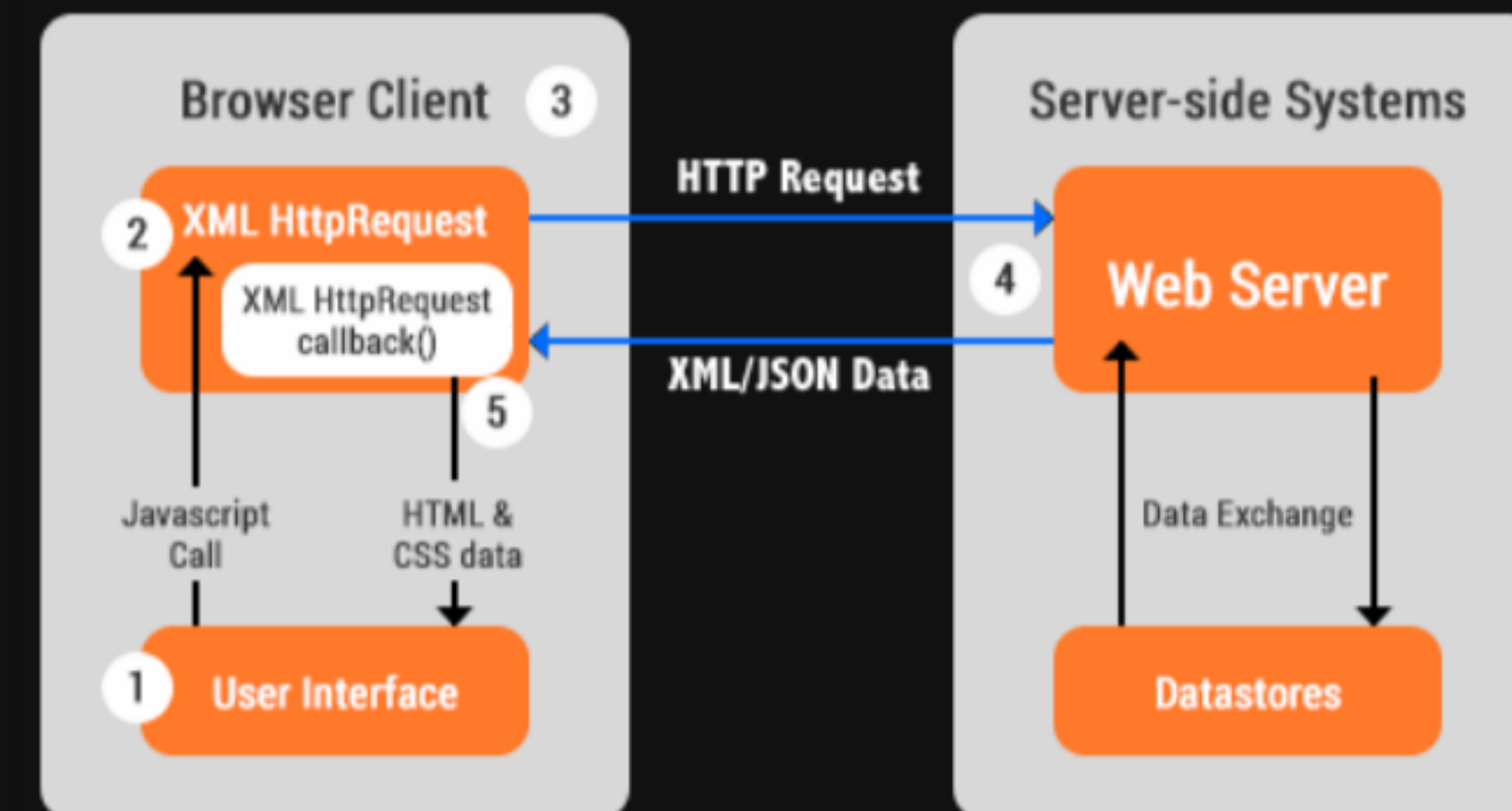
AJAX

AJAX



Asynchronous JavaScript and XML refers to a technique for creating better, faster and more interactive client-side web applications

- JavaScript for interacting with the browser and responding to events
- The DOM for accessing and manipulating the structure of the HTML page
- XML, which represents the data exchanged between the server and client
- An XMLHttpRequest object for asynchronously exchanging data between the client and the server



Not a technology in itself, AJAX refers to a variety of technologies coming together

XMLHttpRequest

The **XMLHttpRequest** object provides the API to make AJAX requests, using a variety of formats within a JavaScript application.

```
var ajax;

if (window.XMLHttpRequest) {
    // Mozilla, Safari, IE7+ ...
    ajax = new XMLHttpRequest();
} else if (window.ActiveXObject) {
    // IE 6 and older
    ajax = new ActiveXObject('Microsoft.XMLHTTP');
}

// run this when the ajax request completes
ajax.onreadystatechange = function() {
    if (ajax.readyState === 4 && ajax.status === 200) {
        alert(ajax.responseText);
    }
};

// start the AJAX request
ajax.open('GET', 'http://localhost:8080/api/users', true);
ajax.setRequestHeader('Content-type', 'application/json');
ajax.send();
```

{ Exercise }

Fetch Customer data with AJAX

Load the customers from the JavaBank REST API
using the XMLHttpRequest object